

Step 1: Generate launch certificate public-private key pair:

```
[root@node_242 ca]# openssl genrsa -out rootCA.key 2048
Generating RSA private key, 2048 bit long modulus
.....+++
e is 65537 (0x10001)
[root@node_242 ca]#
[root@node_242 ca]# ls
rootCA.key
[root@node_242 ca]#
[root@node_242 ca]# openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 365000 -out rootCA.pem
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [XX]:CN
State or Province Name (full name) []:IN
Locality Name (eg, city) [Default City]:newyork
Organization Name (eg, company) [Default Company Ltd]:cpplus
Organizational Unit Name (eg, section) []:server
Common Name (eg, your name or your server's hostname) []:dmp-test-ca.cppluscloud.com
Email Address []:aws@gmail.com
您在 /var/spool/mail/root 中有邮件
[root@node_242 ca]#
[root@node_242 ca]# ls
rootCA.key  rootCA.pem
[root@node_242 ca]#
[root@node_242 ca]# cat >> dmp.ext <<EOF
> authorityKeyIdentifier=keyid,issuer
> basicConstraints=CA:FALSE
> keyUsage=digitalSignature, nonRepudiation, keyEncipherment, dataEncipherment
> subjectAltName=@alt_names
>
> [alt_names]
> DNS.1 = dmp-test-ca.cppluscloud.com

[root@node_242 ca]# ls
dmp.ext  rootCA.key  rootCA.pem
[root@node_242 ca]# openssl req -new -sha256 -nodes -out dmp-test-ca.cppluscloud.com.csr -newkey rsa:2048 -keyout dmp-test-ca.cppluscloud.com.key
Generating a 2048 bit RSA private key
.....+++
writing new private key to 'dmp-test-ca.cppluscloud.com.key'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [XX]:IN
State or Province Name (full name) []:in
Locality Name (eg, city) [Default City]:haiya
Organization Name (eg, company) [Default Company Ltd]:cp
Organizational Unit Name (eg, section) []:cp
Common Name (eg, your name or your server's hostname) []:dmp-test-ca.cppluscloud.com
Email Address []:cp@gmail.co

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:chdgdffree
An optional company name []:
您在 /var/spool/mail/root 中有邮件
[root@node_242 ca]#
[root@node_242 ca]# ls
dmp-test-ca.cppluscloud.com.csr  dmp-test-ca.cppluscloud.com.key  rootCA.key  rootCA.pem
[root@node_242 ca]#
[root@node_242 ca]# openssl x509 -req -in dmp-test-ca.cppluscloud.com.csr -CA rootCA.pem -CAkey rootCA.key -CAcreateserial -out dmp-test-ca.cppluscloud.com.crt -days 36500 -sha256
extfile dmp.ext
Signature ok
subject=C=IN/ST=in/L=haiya/O=cp/OU=cp/CN=dmp-test-ca.cppluscloud.com/emailAddress=cp@gmail.co
Getting CA Private Key
```

Step 2: Configure the key pair in the Nginx server:

```
hash $remote_addr;
server 172.31.31.70:8083 max_fails=2 fail_timeout=30s;
server 172.31.17.212:8083 max_fails=2 fail_timeout=30s;
}

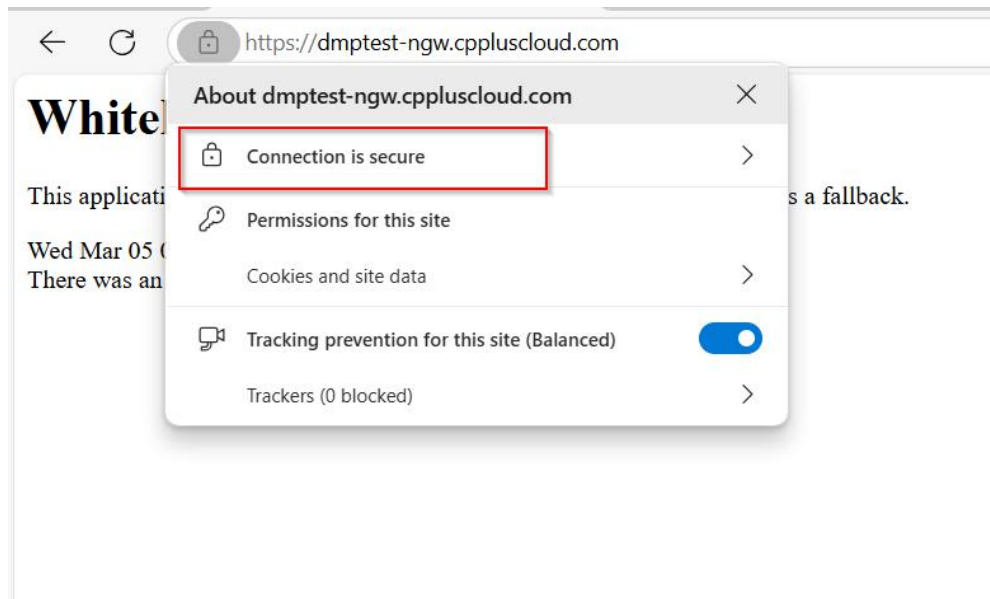
server {
    listen 8083;
    proxy_pass ws_backend;
    proxy_buffer_size 4k;
    #ssl_handshake_timeout 15s;
    #ssl_certificate /etc/emqx/certs/cert.pem;
    #ssl_certificate_key /etc/emqx/certs/key.pem;
}

upstream wss_backend {
    zone tcp_servers 64k;
    hash $remote_addr;
    server 172.31.31.70:8084 max_fails=2 fail_timeout=30s;
    server 172.31.17.212:8084 max_fails=2 fail_timeout=30s;
}

server {
    listen 8084;
    proxy_pass wss_backend;
    proxy_buffer_size 4k;
    #ssl_handshake_timeout 15s;
    ssl_certificate /etc/letsencrypt/live/65-2-183-232.kp2p.cppluscloud.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/65-2-183-232.kp2p.cppluscloud.com/privkey.pem;
}

[root@ip-172-31-22-215 65-2-183-232.kp2p.cppluscloud.com]# ll
total 8
-rw-r--r-- 1 root root 3368 Apr 24 2024 fullchain.pem
-rw----- 1 root root 227 Apr 24 2024 privkey.pem
[root@ip-172-31-22-215 65-2-183-232.kp2p.cppluscloud.com]#
```

Step 3: Obtain the communication certificate:



Step 4: During the production of the device, burn the launch certificate's public key into the device.

Step 5: Trun on the device

- Packet capture shows that the device successfully retrieves the communication certificate using the HTTPS protocol.
- Packet capture shows that the device has connected to the relay server using the HTTPS protocol.

Step 6: Set up upgradable firmware for the device, and initiate the firmware upgrade process. It can be observed that the device successfully upgraded the firmware.

- - Packet capture shows that the device successfully downloaded the firmware using the HTTPS protocol.

Code Snippet:

```

ssl_context_t *sslctx = malloc(sizeof(ssl_context_t));
if(NULL == sslctx)
{
    sslerr = -1;
    log_info(KP2P_SSL_CATALOG,"out of memory to init ssl\n");
    return NULL;
}

mbedtls_debug_set_threshold(1);
mbedtls_net_init(&sslctx->server_fd);
mbedtls_ssl_init(&sslctx->ssl);
mbedtls_ssl_config_init(&sslctx->conf);
mbedtls_ctr_drbg_init(&sslctx->ctr_drbg);
mbedtls_entropy_init(&sslctx->entropy);
sslctx->rtinit = 0;
int ret = -1;
const unsigned char *pers="jdmppsk";
char remport[8]={0};
sprintf(remport,"%d",port);

if(( ret = mbedtls_ctr_drbg_seed(&sslctx->ctr_drbg, mbedtls_entropy_func, &sslctx->entropy,
                                (const unsigned char *)pers,
                                strlen( pers ))) != 0 )
{
    log_info(KP2P_SSL_CATALOG, " failed\n ! mbedtls_ctr_drbg_seed returned %d\n", ret );
    sslerr = -2;
    KP2P_Ssl_DeInit(&sslctx);
    return NULL;
}

ret = mbedtls_net_connect(&sslctx->server_fd, host,remport, MBEDTLS_NET_PROTO_TCP);
if(ret!=0)
{
    log_info(KP2P_SSL_CATALOG,"mbedtls_net_connect %d %s:%s\n",ret,host,remport);
    sslerr = -3;
    KP2P_Ssl_DeInit(&sslctx);
    return NULL;
}

```

```

if ( ret = mbedtls_ssl_config_defaults(&sslctx->conf,
                                      MBEDTLS_SSL_IS_CLIENT,
                                      MBEDTLS_SSL_TRANSPORT_STREAM,
                                      MBEDTLS_SSL_PRESET_DEFAULT ) ) != 0 )
{
    log_info(KP2P_SSL_CATALOG, " failed\n ! mbedtls_ssl_config_defaults returned %d\n", ret );
    sslerr = -2;
    KP2P_Ssl_DeInit(&sslctx);
    return NULL;
}

if(ca && (calen >0)){
    mbedtls_x509_crt_init(&sslctx->cert);
    ret = mbedtls_x509_crt_parse(&sslctx->cert, (const unsigned char *)ca, calen + 1);
    log_info(KP2P_SSL_CATALOG," mbedtls_x509_crt_parse returned %d\n", ret );

    mbedtls_ssl_conf_ca_chain(&sslctx->conf, &sslctx->cert, NULL);
    sslctx->rtinit = 1;

    if(verify){
        mbedtls_ssl_conf_authmode( &sslctx->conf, MBEDTLS_SSL_VERIFY_REQUIRED );
    }
    else{
        mbedtls_ssl_conf_authmode( &sslctx->conf, MBEDTLS_SSL_VERIFY_NONE );
    }
}

mbedtls_ssl_conf_rng( &sslctx->conf, mbedtls_ctr_drbg_random, &sslctx->ctr_drbg );
mbedtls_ssl_conf_dbg( &sslctx->conf, mbedtls_Debug, stdout );
mbedtls_ssl_conf_min_version( &sslctx->conf, MBEDTLS_SSL_MAJOR_VERSION_3, MBEDTLS_SSL_MINOR_VERSION_3);
mbedtls_ssl_conf_max_version( &sslctx->conf, MBEDTLS_SSL_MAJOR_VERSION_3, MBEDTLS_SSL_MINOR_VERSION_3);
mbedtls_ssl_conf_read_timeout( &sslctx->conf, 30000);
mbedtls_ssl_set_bio(&sslctx->ssl, &sslctx->server_fd, mbedtls_net_send, mbedtls_net_recv, mbedtls_net_recv_timeout );
if( ( ret = mbedtls_ssl_setup( &sslctx->ssl, &sslctx->conf ) ) != 0 )
{
    log_info(KP2P_SSL_CATALOG, " failed\n ! mbedtls_ssl_setup returned %d\n", ret );
    sslerr = -3;
    KP2P_Ssl_DeInit(&sslctx);
    return NULL;
}

```

Flowchart:

